MyCodeSchool

Code review — adrian-design-patterns-java

Runde de review pe proiectul tău, cea mai nouă sus.

Review pe exercițiile de design patterns (ex1 Strategy + Decorator bonus, ex2 Observer). ex3–ex6 sunt încă starter, deci nu intră în rundă.

Runda 1 — 2026-10-23 — sha 2f3db33

Poarta de compilare: `javac` pe toate cele 33 de fișiere `.java` trece (OpenJDK 25). Testare3–6 rulează fără eroare, pentru că sunt goale.

🔴 Critice

B1 🔴 Ex1: validarea e demonstrată fără try/catch, excepția oprește programul, iar Testare1 a fost comentat în Main

`src/app/ex1/Testare1.java:20`

CUM E ACUM: `Testare1.java:20` are `Comanda comanda2 = new Comanda("Alex", 0, 100, new LivrareStandard());` fără try/catch, cu o variabilă nefolosită. Ca să nu mai vadă crash-ul, ai comentat `Testare1.ruleaza();` în `Main.java:12`, deși Main era „nu-l modifici”. La Run ‘Main.main()’ nu mai apare nimic din ex1, deci lipsesc cele 4 rânduri cerute. Rândul `Weight must be positive` nu se afișează nici cu Main-ul original.

Repro (sondă în scratchpad, în ordinea din Main-ul original):

```
Standard: 20.0
Express: 47.0
Gratuita: 0.0
Exception in thread "main" java.lang.IllegalArgumentException: Weight must be positive
    at app.ex1.models.Comanda.<init>(Comanda.java:11)
    at app.ex1.Testare1.ruleaza(Testare1.java:20)
```

Exit code 1, Testare2 nu mai rulează. Cu `java -cp out app.Main` (Main-ul comis) nu apare nimic din ex1, primul rând e `Plasata`.

DE CE: O excepție aruncată de constructor urcă pe stiva de apeluri: `Comanda.<init>`, apoi `Testare1.ruleaza`, apoi `Main.main`. Pe fiecare nivel se caută un `catch` potrivit. Dacă nu găsește niciunul, JVM-ul oprește firul `main`, afișează `Exception in thread "main"` cu stack trace-ul și iese cu codul 1. Așa se vede în rulare: după `Gratuita: 0.0` vine direct trace-ul, iar Testare2 nu mai pornește. Validarea din constructor funcționează, problema e doar la cine prinde excepția. Cerința cere ca Testare1 să o prindă singur (try/catch + `ex.getMessage()`), ca programul să continue. Când comentezi apelul din Main, ascunzi simptomul și pierzi tot ex1.

FIX: în Testare1, la rândul 20:

```
try {
    new Comanda("Ion", 0, 50, new LivrareStandard());
} catch (IllegalArgumentException ex) {
    System.out.println(ex.getMessage());
}
```

În `Main.java:12` pui la loc `Testare1.ruleaza();`. Testare3–6 sunt goale și rulează fără eroare, deci pot fi și ele decomentate, ca în starter.

+ același bug la: - `src/app/Main.java:11` — `// Testare1.ruleaza();` comentat, deci ex1 nu rulează din Main; cauza e linia 20 din Testare1. Fix la sursă: try/catch în Testare1.java:20, apoi Testare1 se decomentează în Main. - `src/app/Main.java:13-16` — Testare3..Testare6 sunt comentate, dar corpurile lor sunt goale (`Testare3.java:5-6`, `Testare4.java:5-6`, `Testare5.java:13-14`, `Testare6.java:12-13`). Când le vei completa, validările din ex6 (`NotificatorSms.java:14,17`; `CuPrefix.java:9,12`; `CuTrunchiere.java:9,12`; `CuJurnal.java:8`; `NotificatorEmail.java:13`) și din ex5 (`Utilizator.java:9,12`; `Student.java:9`; `Profesor.java:9`; `IncarcatorUtilizatori.java:10`) vor ridica aceeași problemă dacă le demonstrezi fără try/catch. Acum nu se încalcă nicăieri, pentru că nu există apeluri. - `src/app/ex1/models/Comanda.java:46-48` — verificarea `greutateKg <= 0` din `costTransport()` e cod mort, duplicat al celei din constructor (liniile 10-12), pentru că greutatea e `final` și constructorul a respins deja valorile invalide. Nu e aceeași încălcare, dar are aceeași cauză: validarea nu e demonstrată într-un loc controlat. - `src/app/ex1/Testare1.java:20` — singura demonstrație de validare fără try/catch din repo. În ex2 e corect la `Testare2.java:31-35`; `Testare2.java:38` (`schimbaStare("RETURNAT")`) e valid și nu aruncă. Nu există alte apeluri invalide neprotejate.

B2 🔴 Bonus Decorator: întoarce reducerea, nu costul redus

`src/app/ex1/models/LivrareCuReducere.java:24`

CUM E ACUM: `double costRedus = costTotal * procent / 100; return costRedus;` întoarce doar partea scăzută din cost. Cerința bonusului era costul strategiei interioare minus procentul.

Repro (sondă în scratchpad):

```
Comanda k = new Comanda("Ana", 10, 100, new LivrareStandard());
k.schimbaStrategie(new LivrareCuReducere(new LivrareStandard(), 10));
k.costTransport()                                  -> REDUCERE 10%: 2.0   (corect 18.0)
LivrareCuReducere(new LivrareStandard(), 0)        -> 0.0                 (corect 20.0)
LivrareCuReducere(new LivrareExpress(), 25)        -> 11.75               (corect 35.25)
```

DE CE: `costTotal * procent / 100` calculează felia care se scade, adică 10% din 20 = 2. Costul care rămâne e restul: `costTotal - felia`, adică `costTotal * (1 - procent / 100)`. Rularea arată și testul simplu pentru orice Decorator: la 0% ar trebui să fie transparent și să dea exact costul interior (20.0), dar dă 0.0. Un decorator împachetează comportamentul interior și îl modifică. Nu trebuie să-l înlocuiască cu altă mărime. Aici „livrare cu 10% reducere" costă de 10 ori mai puțin decât ar trebui. Structura e corectă (ține altă `LivrareStrategie` și îi deleagă calculul), greșită e doar formula.

FIX: `return costTotal - costTotal * procent / 100;` (sau `costTotal * (1 - procent / 100)`).

+ același bug la: - Nu am găsit alte locuri cu aceeași greșeală. Am căutat în tot `src` (`procent`, `%`, `/ 100`, `* cost`, `pret|discount|taxa|tva|adaos|comision`) și singura potrivire de calcul procentual e `src/app/ex1/models/LivrareCuReducere.java:24`. - `LivrareCuReducere` nu e folosită nicăieri altundeva: nici în `Testare1.java`, nici în Main sau în ex2–ex6. Testele tale nu prind bug-ul și nu mai e nimic de aliniat în alte fișiere. - `src/app/ex6/models/CuJurnal.java`, `CuPrefix.java` și `CuTrunchiere.java` sunt alți decoratori, dar sunt încă stub-uri (`throw new UnsupportedOperationException`). Nu încalcă regula, fiindcă nu au logică de calcul.

🟡 Importante

- **M1** `src/app/ex2/Testare2.java:14-38` — scriptul nu e cel din cerință: în plus are `comanda1` cu „Amazon"/„livrat" și abonamentul la „RETURNAT", iar al doilea `schimbaStare("Expediata")` lipsește. Ieșirea are 12 rânduri, cerința are 7, și nu se vede că o stare identică nu anunță pe nimeni. Logica din Comanda e corectă: cu scriptul exact din cerință, rulat în sondă, ieșirea e identică cu cea așteptată. Experimentele merg într-o metodă separată sau după scriptul cerut.
- **M2** `src/app/ex1/models/Comanda.java:46-48` — verificarea `greutateKg <= 0` din `costTransport` e cod mort, pentru că `greutateKg` e `final` și constructorul a validat-o deja. În plus, mesajul e în română („Greutatea trebuie sa fie..."), contra constrângerii „mesaje în engleză". Se șterge.
- **M3** `src/app/ex1/models/Comanda.java:38-39` — `schimbaStrategie(null)` e acceptat, iar eroarea apare abia la `costTransport` ca NPE (`because "this.strategie" is null`,

`Comanda.java:50`, reprodus). Trebuie validat la intrare: `IllegalArgumentException("Strategy cannot be null")`. Aceeași regulă și pentru parametrul `strategie` din constructor.

🟢 Cleanups

- **C1** Comentarii lăsate în cod: `Testare2.java:12` (`//CTRL + Alt + L`), `ex2/Comanda.java:47,49` (`//update the state...`), plus `// throw new UnsupportedOperationException` rămase comentate în `ex1/Comanda.java:41`, `ex2/Comanda.java:30,52`, `JurnalLivrare.java:8`, `NotificatorEmail.java:8`, `PanouDepozit.java:9`. Se șterg.
- **C2** Commit-ul `2f3db33` adaugă `.idea/` (`workspace.xml`, `shelf/*.patch`) și un al doilea modul, `src/design-patterns-java.iml`, care se suprapune cu cel din rădăcină. Trebuie scoase din git, iar `.idea/` adăugat în `.gitignore`.
- **C3** Formatare: `implements LivrareStrategie{` (`LivrareCuReducere:3`), `return costRedus;` (`:26`), `if(` / `}\nelse if(` (`ex2/Comanda:40,43`). Se rezolvă cu Ctrl+Alt+L, scurtătura pe care ai notat-o deja.
- **C4** `LivrareCuReducere`: câmpurile `strategie` și `procent` pot fi `final`, iar `getNume()` = "REDUCERE" poate include strategia interioară (ex. `strategie.getNume() + " -" + procent + "%"`).
- **C5** Testare1: `comanda1` / `comanda2` → `comanda`. Variabila `comanda2` nici nu e folosită.
- Bine: în ex1, `Comanda` deleagă pur strategiei, fără niciun if/switch pe tip, iar formulele dau exact 20.0 / 47.0 / 0.0.
- Bine: în ex2, `Comanda` cunoaște doar `Observator`. Ordinea validare → stare egală → setare → for pe observatori e exact cea cerută, iar scriptul din cerință produce ieșirea identică.
- Bine: `LivrareCuReducere` are forma corectă de Decorator (împachetează o `LivrareStrategie` și îi deleagă calculul), doar aritmetica e greșită. Experimentul cu al doilea `NotificatorEmail` abonat arată corect că emailul vine de două ori.

Before / After (🔴)

#	Before	After
B1	`Comanda comanda2 = new Comanda("Alex", 0, 100, new LivrareStandard());` în `Testare1.java:20`, iar în `Main.java:12` `// Testare1.ruleaza();`	`try { new Comanda("Ion", 0, 50, new LivrareStandard()); } catch (IllegalArgumentException ex) { System.out.println(ex.getMessage()); }` și `Testare1.ruleaza();` pus la loc în Main
B2	`double costRedus = costTotal * procent / 100;` `return costRedus;`	`return costTotal - costTotal * procent / 100;`

Q&A

1. Cerința ex1 cere să afișezi `ex.getMessage()` pentru `new Comanda("Ion", 0, 50, ...)`. Pe ce drum merge excepția dacă nu o prinzi în Testare1? De ce dispare atunci și tot ce urma după ea, inclusiv Testare2?
2. `LivrareCuReducere(new LivrareStandard(), 0)` ar trebui să coste exact cât `LivrareStandard`. Ce îți spune regula asta despre orice Decorator? Ce dă codul tău acum și de ce?
3. În ex2, de ce verifici `stareNoua.equals(stare)` ÎNAINTE să setezi `stare = stareNoua`? Ce s-ar afișa la al doilea `schimbaStare("Expediata")` dacă ai inversa cele două rânduri?

Starea rundelor anterioare

Prima rundă pe acest proiect: nu există `CODE_REVIEW.md` anterior, deci nu sunt 🔴 de verificat. Elevul a rezolvat doar ex1 și ex2; ex3–6 sunt încă starter.